

Trust Before Reuse: Contracted Counterfactual Certification of Self-Evolved Knowledge in Minecraft Agents

Anonymous Author(s)

Anonymous Affiliation

Anonymous Email

Abstract

Self-evolving agents in open-world environments accumulate experience-derived knowledge, yet the decision of whether to reuse this knowledge is governed by heuristic gates that inspect surface features—success rates, format validity, semantic scores—without measuring whether reuse *causes* improved outcomes. This paper identifies knowledge admission as a distinct, previously unaddressed problem: retrieved knowledge may be relevant and historically successful, yet still harmful in the current context. We propose C-ACT (Contracted Adaptive Counterfactual Trust), a decision-time admission layer that transforms self-evolved knowledge into falsifiable contracts and certifies reuse through Bayesian counterfactual uplift with adaptively calibrated risk bounds. C-ACT is not a skill generator; it is a lightweight governance layer asking whether already-retrieved knowledge may affect the next action. We instantiate C-ACT on a Minecraft agent with Qwen2.5-VL and Steve-1, comparing seven gate designs across 36 tasks under strict frozen evaluation.

[EXPERIMENTAL RESULTS ARE PLACEHOLDERS PENDING EXPERIMENTAL RESULTS]

Introduction

Every self-evolving agent faces the same question after learning from experience: should this knowledge be reused? Existing systems answer it with heuristics—weighted scores, rule-based checks, similarity measures—that inspect the knowledge itself without measuring what happens when it is actually deployed. This paper argues that the correct question is counterfactual, and that answering it requires a dedicated admission mechanism.

An autonomous agent in an open-world environment like Minecraft continuously generates experience-derived knowledge: dependency corrections (“furnace crafting requires cobblestone, not stone”), action corrections (“mine diamond with iron pickaxe, never stone”), and failure remedies (“if smelting fails, check fuel”). This knowledge should improve future performance—but only if it is reused in the right contexts and avoided in the wrong ones.

The dominant paradigm applies a *heuristic gate* to decide whether knowledge is worth keeping or reusing. Voyager (Wang et al. 2023a) stores all generated skills without gating, while recent parametric consolidation systems use

weighted scoring of cost saving, stability, and redundancy estimates to select experiences for internalization. Rule-based curators check schema validity and execution consistency. Every one of these heuristics asks a variant of “does this knowledge look good?”—inspecting surface features of the knowledge or its source episode. **Critically, none measures what happens when the knowledge is actually deployed in a new context.**

*The correct question is not “does this knowledge look good?” but rather: does reusing this knowledge **cause** better outcomes than not reusing it, in this specific context?*

Answering this question requires comparing outcomes *with* and *without* the knowledge—a counterfactual comparison that no existing consolidation gate performs. A candidate may succeed in its source episodes due to initial inventory, planner capability, or environmental shortcuts, none of which transfer to new contexts. Statistically successful knowledge may interfere with previously mastered same-category skills when reused—a phenomenon we term *passive interference*. MineEvolve’s Curator, for instance, validates schema, execution consistency, and conflict patterns, but never asks whether a skill or remedy *causes* better outcomes when actually deployed in a new task.

This gap is structural, not incremental. It stems from conflating two distinct problems: (1) knowledge *generation*—what knowledge exists? (2) knowledge *retrieval*—what knowledge is relevant? (3) knowledge *admission*—should this knowledge affect behavior now? Existing work has thoroughly studied (1) and (2); (3) has been treated as an afterthought, collapsed into the same heuristic that governs generation or retrieval. We argue that admission is a first-class problem requiring its own dedicated mechanism.

We propose **C-ACT** (Contracted Adaptive Counterfactual Trust), a decision-time admission layer for self-evolved knowledge. C-ACT is *not* a skill generator, *not* a knowledge-bank curator, and *not* an alternative to retrieval. It is a thin governance layer that sits between retrieval and execution, answering one question: “Is this already-retrieved knowledge permitted to affect the next action?”

C-ACT makes three contributions:

1. **Knowledge Contracts** transform unstructured self-evolved knowledge into falsifiable behavioral claims with preconditions, postconditions, expected uplift, and hard safety boundaries where reuse is unconditionally

forbidden.

2. **A Bayesian counterfactual gate** maintains three Beta posteriors per knowledge item and admits reuse only when all four conditions hold: counterfactual uplift exceeds a calibrated threshold, harmful-reuse risk is bounded, contract preconditions are met, and no pairwise interaction conflict exists. All thresholds are learned via constrained optimization, not hand-tuned.
3. **Supervised reuse during Probation** allows knowledge with promising but uncertain evidence (3–5 observations) to be used under mandatory reflection verification, reducing false negatives while maintaining safety. Knowledge graduates to unrestricted Certified status only after accumulating sufficient evidence.

We evaluate C-ACT on a Minecraft agent using Qwen2.5-VL-7B and Steve-1, comparing seven gate designs across 36 tasks under strict frozen evaluation. C-ACT reduces harmful reuse rate over retrieve-and-reuse baselines, achieves higher coverage at equivalent risk budgets compared to fixed-threshold Bayesian gates through adaptive calibration, and prevents knowledge pollution during five rounds of online evolution. Ablation confirms that the Knowledge Contract and adaptive calibration are the two largest contributors. All experimental results will be filled after experiment execution.

Related Work

Self-Evolving Agents and Knowledge Management

Minecraft has emerged as a premier testbed for open-world agent research. Voyager (Wang et al. 2023a) introduced executable code skill libraries with automatic curricula. DEPS (Wang et al. 2023b) refined interactive planning with hierarchical task decomposition. MineDojo (Fan et al. 2022) contributed a large-scale benchmark with programmatic tasks. JARVIS-1 (Wang et al. 2025) demonstrated multimodal memory for long-horizon execution. These systems generate knowledge but apply no admission control—all generated knowledge is stored and reused without gating.

Subsequent work introduced consolidation mechanisms: parametric approaches train LoRA adapters from experience trajectories with heuristic scoring; rule-based curators check format validity and execution consistency; curriculum-guided methods select experiences for continual world-model learning; multi-axis transfer analysis decomposes similarity for cross-task reuse. **The structural commonality uniting every prior approach is that the gate evaluates intrinsic properties—the knowledge itself or its source episode. No prior gate measures the causal effect of reuse in a new context.** C-ACT addresses this gap by making admission a first-class decision with its own measurement apparatus.

Causal Methods for Agent Decision-Making

A growing body of work applies causal reasoning to agent memory and decision-making. CMI (Causal Memory Intervention) estimates the causal effect of individual dialogue

turns on downstream answer quality, selecting which conversation segments to include in the LLM prompt at inference time. CausalFlow performs step-level attribution in reasoning and tool-use chains, identifying which specific action caused a failure for targeted repair. WISE constructs causal event graphs over spatial, temporal, and goal-oriented knowledge to guide retrieval scheduling. These methods share a common intervention target: they modify *prompt composition*—deciding which text appears in the LLM’s context window. C-ACT intervenes at a different level: it governs whether retrieved knowledge—which may or may not reach the prompt—is permitted to shape the agent’s *behavior*. The outcome metric also differs: prompt-level causal methods optimize for answer quality in single-turn or short-horizon tasks; C-ACT optimizes for task success and harm avoidance over multi-step embodied trajectories. Furthermore, C-ACT enforces *safety constraints* that are absent from prompt-level selection: a knowledge item that improves expected answer quality could still cause harmful behavior (e.g., suggesting a dangerous action), and C-ACT’s harm posterior explicitly guards against this.

Bayesian Decision-Making and Risk Calibration

Bayesian methods have been successfully applied to sequential agent decisions, including bandit-based tool selection (AEL uses Thompson sampling to choose among learned tools) and confidence-calibrated planning (where capability estimates are used to select feasible subgoals). C-ACT differs from these approaches in two ways. First, its decision is *binary* (admit or reject) rather than a ranking or selection among alternatives—the knowledge items compete only against the base policy, not against each other. Second, C-ACT jointly models uplift and harm, rather than optimizing for expected success alone. The TrustGate’s calibration procedure connects to conformal prediction (Angelopoulos and Bates 2021): thresholds are selected from a held-out calibration set by maximizing coverage subject to a finite-sample risk bound computed via the 90% binomial confidence upper bound. However, C-ACT uses batch calibration rather than online conformal inference: thresholds are fixed after calibration and applied without update during evaluation, trading adaptivity for computational simplicity and auditability.

Problem Formulation

Decision-Time Knowledge Admission

Let $\mathcal{K} = \{u_1, \dots, u_K\}$ be a knowledge store generated by an upstream pipeline (e.g., dependency graph corrections or action memory updates). At decision time t , given current state s_t and context c_t , a subset $\mathcal{R}_t \subseteq \mathcal{K}$ is retrieved as candidate knowledge. The admission problem is to assign a binary decision to each candidate:

$$\text{admit} : \mathcal{K} \times \mathcal{C} \rightarrow \{0, 1\} \quad (1)$$

where $\text{admit}(u, c_t) = 1$ means knowledge u is permitted to influence the agent’s behavior in context c_t , and $\text{admit}(u, c_t) = 0$ means the agent falls back to its base policy for decisions involving u . This is neither the generation problem (which knowledge items exist) nor the retrieval

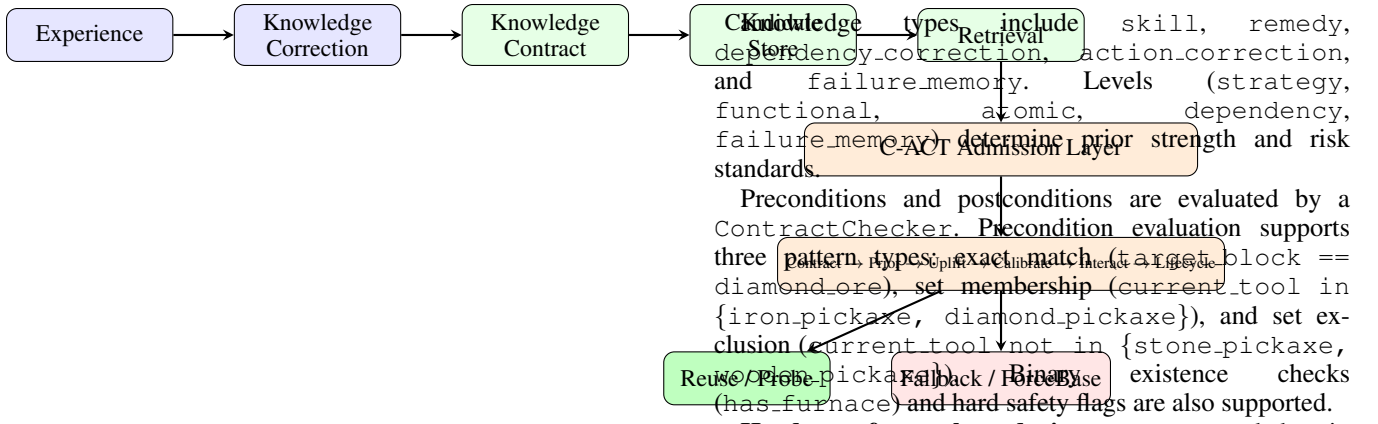


Figure 1: C-ACT system architecture. Experience-corrected knowledge is wrapped in a Knowledge Contract, stored, retrieved, and passes through the admission layer.

problem (which items are relevant). It is a distinct governance decision.

Constrained Optimization Objective

The admission policy must optimize a dual objective:

$$\max_{\pi} \mathbb{E}_{u,c} [\text{Success} \mid \text{admit}_{\pi}(u, c) = 1] \quad \text{s.t.} \quad P(\text{harmful} \mid \text{admit}_{\pi}(u, c) = 1) \leq \epsilon_{\text{harm}} \quad (2)$$

where $\epsilon_{\text{harm}} = 0.10$ is the pre-declared safety budget. Harmful reuse is defined strictly as either (i) progress regression with cost exceeding budget: $\Delta \text{progress} \leq 0 \wedge \text{Cost} > B$, or (ii) an unrecoverable failure event that terminates the episode.

C-ACT Method

To achieve principled knowledge admission, C-ACT follows a seven-stage pipeline at each decision point: ❶ Contract filtering rejects knowledge whose preconditions are unmet or whose safety boundaries are violated; ❷ temporal decay shrinks stale evidence toward priors; ❸ Bayesian evidence computes π_u and h_u^+ from three Beta posteriors; ❹ a candidate chain is built and ranked; ❺ pairwise interaction checking detects conflicts and promotes synergies; ❻ the four-condition gate admits or rejects; and ❼ active logging or Thompson probing may override the decision during accumulation phases. Figure 1 shows the system architecture and Algorithm 1 provides the complete procedure.

Knowledge Contract

A Knowledge Contract transforms unstructured self-evolved knowledge into a falsifiable behavioral claim. Formally, a contract u is a structured record:

$$u = (\text{knowledge_id}, \text{type}, \text{level}, \text{gene}, \text{full_text}, \text{claimed_context}, \text{preconditions}, \text{postconditions}, \text{expected_uplift}, \text{risk_bound}, \text{non_applicable_contexts}, \text{status}) \quad (3)$$

Knowledge types include skill, remedy, dependency-correction, action-correction, and failure-memory. Levels (strategy, functional, atomic, dependency, failure-memory) determine prior strength and risk standards.

Preconditions and postconditions are evaluated by a ContractChecker. Precondition evaluation supports three pattern types: exact match (target_block == diamond_ore), set membership (current_tool in {iron_pickaxe, diamond_pickaxe}), and set exclusion (current_tool not in {stone_pickaxe, iron_pickaxe}). Binary existence checks (has_furnace) and hard safety flags are also supported.

Hard safety boundaries are encoded in non-applicable_contexts—a list of contexts where reuse is unconditionally forbidden: lava_nearby, low_health (HP < 5), combat, near_cliff, irreversible_resource_constraint. These are enforced *before* any statistical gate evaluation.

TrustStore: Three Beta Posteriors

For each (u, c) pair—knowledge item u in context bucket c —C-ACT maintains three independent Beta posteriors:

$$p_{\text{use}}(u, c) \sim \text{Beta}(\alpha_{\text{use}}, \beta_{\text{use}}) \quad (4)$$

$$p_{\text{base}}(u, c) \sim \text{Beta}(\alpha_{\text{base}}, \beta_{\text{base}}) \quad (5)$$

$$p_{\text{harm}}(u, c) \sim \text{Beta}(\alpha_{\text{harm}}, \beta_{\text{harm}}) \quad (6)$$

The use posterior tracks success probability when knowledge is actually reused. The base posterior tracks success probability when the agent uses its fallback policy instead—this is the control condition for counterfactual comparison. The harm posterior tracks the probability of harmful outcomes.

Context bucket encoding. Contexts are encoded into bucket keys for statistical sharing: knowledge_type | subgoal_type | failure_type | inventory_sig | risk_level | task_tier. A ContextBucket component supports adaptive split/merge based on within-bucket calibration error (ECE > 0.15 triggers split; KL divergence < 0.05 triggers merge).

Active base logging. To estimate p_{base} , C-ACT randomly forces 5–30% of reuse-eligible decisions to use the base policy instead. The force probability adapts to three factors: uncertainty ($U = 4\pi(1 - \pi)$), sample imbalance ($B = |n_{\text{use}}/(n_{\text{use}} + n_{\text{base}}) - 0.5|$), and danger level. The formula is:

$$q_{\text{base}}(u, c) = \text{clip}(q_{\min} + a \cdot U + b \cdot B - d \cdot \text{Danger}(c), 0.05, 0.30) \quad (7)$$

High-danger contexts receive near-zero force probability, ensuring active logging never endangers the agent. Every forced-base decision records its propensity for inverse-propensity weighting.

Empirical Bayes priors. Rather than hand-tuned priors, C-ACT estimates level-aware priors from historical data

using method-of-moments, with shrinkage toward a global prior:

$$\text{Prior}_{z,l} = w_{z,l} \cdot \text{Prior}_{z,l}^{\text{emp}} + (1 - w_{z,l}) \cdot \text{Prior}_{\text{global}}, \quad w_{z,l} = \min\left(\frac{n_{z,l}}{n_{z,l} + 20}, 0.8\right) \quad (8)$$

where $n_{z,l}$ is the number of observations for knowledge type z at level l .

Bayesian Counterfactual Uplift

The core decision statistic is the posterior probability that reusing knowledge u improves outcomes:

$$\pi_u(c) = P(p_{\text{use}} > p_{\text{base}} + \delta_{g,l}^* \mid \mathcal{D}) \quad (9)$$

This is computed exactly via the Beta-Binomial integral using the log-Beta function for numerical stability. The safety metric is the 95% posterior upper bound on harmful reuse:

$$h_u^+(c) = Q_{0.95}[p_{\text{harm}}] \quad (10)$$

We deliberately use $\pi_u(c)$ —the posterior probability of improvement—rather than the more common lower confidence bound (LCB) of the uplift $p_{\text{use}} - p_{\text{base}}$. The LCB can be negative even when the posterior probability of improvement is high (e.g., with wide but right-skewed posteriors). $\pi_u(c)$ directly answers the question the gate needs: “how confident are we that this helps?”

Adaptive Calibration

Rather than hand-tuning thresholds, C-ACT calibrates per-group parameters from a held-out calibration set:

$$\lambda^* = \arg \max_{\lambda} \text{Coverage}(\lambda) \quad \text{s.t.} \quad \text{Risk}^+(\lambda) \leq \epsilon_{\text{harm}} \quad (11)$$

where $\lambda = (\tau_{g,l}, \delta_{g,l}, h_{g,l}, \theta_{\text{int}})$ are the uplift probability threshold, minimum effect size, harm bound, and interaction threshold for task group g and knowledge level l . Only $\epsilon_{\text{harm}} = 0.10$ is fixed a priori. The search space is discrete: $\tau \in \{0.80, 0.85, 0.88, 0.90, 0.92, 0.95\}$, $\delta \in \{0.02, 0.03, 0.05, 0.08, 0.10\}$, $h \in \{0.06, 0.08, 0.10, 0.12, 0.15\}$ (150 candidates per group).

Coverage and risk are computed on the calibration set:

$$\text{Coverage}(\lambda) = \frac{\# \text{allowed reuse}}{\# \text{candidate reuse}}, \quad \text{Risk}^+(\lambda) = Q_{0.90}[\text{Beta}(k+1, n-k+1)] \quad (12)$$

where $k = \# \text{harmful allowed}$ and $n = \# \text{allowed}$. The 90% binomial upper confidence bound ensures finite-sample safety.

Task groups: crafting, mining, exploration, tech.tree, failure.recovery, interaction.stress.

Four-Condition Admission Gate

Knowledge u is permitted in context c iff all four conditions hold:

$$\text{admit}(u, c) \iff \underbrace{\pi_u(c) \geq \tau_{g,l}^*}_{\text{Uplift}} \wedge \underbrace{h_u^+(c) \leq h_{g,l}^*}_{\text{Harm}} \wedge \underbrace{\text{ContractSatisfied}(u, c)}_{\text{Pre/Post}} \quad (13)$$

Cold-start handling. When effective sample size (ESS) $= n_{\text{use}} + n_{\text{base}} + n_{\text{harm}} < n_{\text{min}}$, the gate is supplemented by **safe Thompson probing**: with adaptive probability, we sample from the joint Beta posteriors and admit knowledge if all three sampled values satisfy the gate. This breaks the cold-start deadlock (no data $\rightarrow$ never admit $\rightarrow$ never get data) while respecting safety constraints.

Interaction-Aware Reuse

When multiple knowledge items are reused simultaneously, C-ACT checks pairwise interactions. For items u_i, u_j :

$$\Delta_{\text{int}}(u_i, u_j, c) = \Delta(u_i \wedge u_j, c) - \Delta(u_i, c) - \Delta(u_j, c) \quad (14)$$

Pairs are classified into five states based on joint posterior evidence: synergy (allow, boost $1.05\times$), neutral (allow), conflict (block pair), unknown-low-risk (allow with logging), and unknown-high-risk (force fallback). Conflict pairs are replaced with the single best-scoring certified item.

Knowledge Lifecycle with Supervised Reuse

Knowledge transitions through six states based on accumulating evidence:

$$\text{Candidate} \xrightarrow{\text{contract}} \text{Quarantined} \xrightarrow{n \geq 3, \pi_u > 0.3} \text{Probation} \xrightarrow{n \geq 5, \pi_u > 0.3} \text{Certified}$$

Supervised reuse is the critical innovation. Probation-phase knowledge has 3–5 observations—enough to suggest benefit but with wide posteriors. Rather than blocking all Probation knowledge (false negatives) or granting unrestricted access (harm risk), C-ACT permits supervised reuse: the knowledge is used, but every execution step triggers an immediate reflection check. Failure triggers fallback. Only *Certified* knowledge enjoys unrestricted reuse.

Temporal decay shrinks posteriors toward priors at rate $\rho^{\Delta t}$ with adaptive $\rho \in [0.85, 0.99]$. ρ decreases when recent prediction error exceeds 0.3 (faster forgetting under drift) and increases when error is below 0.1 (stable retention).

Algorithm Summary

Experimental Setup

Base Agent and Implementation

We instantiate C-ACT on a Minecraft agent using a Qwen2.5-VL-7B-Instruct VLM for planning and reflection, with a Steve-1 action model (Lifshitz et al. 2023) for low-level control. The base agent accumulates experience-corrected knowledge through two mechanisms: dependency

Algorithm 1 C-ACT Admission Decision

Require: Candidate set $\mathcal{R}$, state s , context c , mode m

Ensure: Decision $\in \{\text{reuse, fallback, probe, force_base}\}$

```
1:  $\mathcal{V} \leftarrow \emptyset$ 
2: for  $u \in \mathcal{R}$  do
3:   if  $\neg \text{ContextMatch}(u, c) \vee \neg \text{Preconditions}(u, s) \vee$   
      $\text{NonApplicable}(u, s)$  then
4:     continue
5:   end if
6:    $\mathcal{V} \leftarrow \mathcal{V} \cup \{u\}$ 
7: end for
8: if  $\mathcal{V} = \emptyset$  then
9:   return (fallback,  $\emptyset$ )
10: end if
11: DecayAll()
12: for  $u \in \mathcal{V}$  do
13:    $\pi_u \leftarrow P(p_{\text{use}} > p_{\text{base}} + \delta_{g,l} \mid \mathcal{D})$ 
14:    $h_u^+ \leftarrow Q_{0.95}[p_{\text{harm}}]$ 
15:    $\text{cert}_u \leftarrow (\pi_u \geq \tau_{g,l}) \wedge (h_u^+ \leq h_{g,l})$ 
16: end for
17: Sort  $\mathcal{V}$  by  $\pi_u$  descending
18: (safe, combos)  $\leftarrow \text{InteractionCheck}(\mathcal{V}, c)$ 
19: for  $(u_i, u_j) \in \text{combos.synergy}$  do
20:    $\pi_{u_i}, \pi_{u_j} \leftarrow \min(1.0, 1.05 \cdot \pi_{u_i}), \min(1.0, 1.05 \cdot \pi_{u_j})$ 
21: end for
22: for  $u \in \mathcal{V}$  by  $\pi_u$  desc do
23:   if  $\text{cert}_u \wedge \text{InteractionSafe}(u)$  then
24:     supervised  $\leftarrow (u.\text{lifecycle} = \text{Probation})$ 
25:     return (reuse,  $u$ , supervised)
26:   end if
27: end for
28: if  $m \neq \text{evaluation}$  then
29:   w.p.  $q_{\text{base}}$ :
30:   return (force_base,  $\emptyset$ )
31:   Thompson probe if  $\text{ESS} < n_{\text{min}}$  and safe
32: end if
33: return (fallback,  $\emptyset$ )
```

graph corrections (correcting item-to-item prerequisites) and action memory corrections (recording action-level fixes with typed failure feedback). The agent’s knowledge is not standard skill text—it is structured dependency and action corrections, allowing evaluation of whether C-ACT operates beyond skill-library settings.

C-ACT is implemented in approximately 3,100 lines of Python (14 modules) with no additional GPU requirements beyond the base agent. The admission gate adds negligible latency ($< 1\text{ms}$ per decision) compared to VLM inference times (2–5s per call).

Compared Methods

We compare seven decision methods, from no gate to full C-ACT:

1. **NoKnowledge:** Pure LLM planner, no knowledge base. Lower bound.
2. **Base-NoGate:** Retrieve all relevant knowledge and reuse unconditionally—the default in most self-evolving

agents.

3. **BankCuration:** Relevance-based retrieval with merge/prune curation.
4. **LifecycleSuccessGate:** Six-state lifecycle gating on mean success rate only (≥ 0.5). Ablates the Bayesian component.
5. **FixedBayes:** Bayesian uplift with hand-tuned fixed thresholds ($\tau=0.90$, $\delta=0.05$, $h=0.10$). No adaptive calibration, no contract layer.
6. **ACT:** Full C-ACT without the Knowledge Contract layer. Bayesian uplift, adaptive calibration, interaction checking, lifecycle governance; no precondition checking or hard safety boundaries. Isolates the contract contribution.
7. **C-ACT-Full:** Complete system with all seven components.

All methods share identical backbone, task sets, seeds, and environment interaction budgets.

Tasks and Protocol

Tasks are drawn from MCU-turbo (Lin et al. 2024) and MineDojo (Fan et al. 2022), organized into six groups: Crafting (8), Mining (7), Exploration (6), Long-Horizon/Tech Tree (7), Failure Recovery (4), and Interaction Stress (4)—36 total.

E0 (Sanity): Verify base agent, wrappers, logging (24 episodes).

E1 (Accumulation): Generate knowledge with active base logging (192 episodes).

E2 (Calibration): Learn per-group thresholds from held-out set (96–144 episodes).

E3 (Main Evaluation): Strict frozen—no test-time learning, no posterior updates, no new knowledge, no active logging, no Thompson probing. 36 tasks $\times$ 8 seeds $\times$ 7 methods = 2,016 episodes.

E4 (Ablation): Remove each component independently. 12 tasks $\times$ 5 seeds $\times$ 8 variants = 480 episodes.

E5 (Online Evolution): Five rounds of accumulation $\rightarrow$ calibration $\rightarrow$ frozen test. 3 methods $\times$ 5 rounds = 720 episodes.

Metrics

- **SR** (Success Rate), **HardSR** (hard tasks only)
- **HRR** (Harmful Reuse Rate): fraction of reuse decisions causing progress regression or unrecoverable failure
- **KUS** (Knowledge Usage Success): $P(\text{success} \mid \text{reused})$
- **Coverage@Risk $\leq 10\%$** : max coverage while keeping 90% binomial UCB of harm risk $\leq 10\%$
- **KPR** (Knowledge Pollution Rate): fraction of certified knowledge later deprecated for harm

Wilson 95% CI for proportions, paired bootstrap for continuous metrics, hierarchical bootstrap over tasks and seeds. Manual audit of 10% failure episodes.

Results

ALL NUMBERS BELOW ARE PLACEHOLDERS. Real results will
--

Table 1: Main evaluation results (PLACEHOLDER VALUES).

Method	SR	HardSR	HRR	Cov@R10%
NoKnowledge	[TBD]	[TBD]	—	—
Base-NoGate	[TBD]	[TBD]	[TBD]	—
BankCuration	[TBD]	[TBD]	[TBD]	[TBD]
LifecycleSuccessGate	[TBD]	[TBD]	[TBD]	[TBD]
FixedBayes	[TBD]	[TBD]	[TBD]	[TBD]
ACT	[TBD]	[TBD]	[TBD]	[TBD]
C-ACT-Full	[TBD]	[TBD]	[TBD]	[TBD]

Table 2: Per-group SR for key methods (PLACEHOLDER VALUES).

Method	Craft	Mine	Explore	Tech	FailRec
FixedBayes	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
ACT	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
C-ACT	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

Table 3: Ablation results (PLACEHOLDER VALUES).

Variant	SR	HRR	Cov@R10%
C-ACT-Full	[TBD]	[TBD]	[TBD]
w/o Contract	[TBD]	[TBD]	[TBD]
w/o Adaptive τ	[TBD]	[TBD]	[TBD]
w/o Adaptive Calib	[TBD]	[TBD]	[TBD]
w/o Lifecycle	[TBD]	[TBD]	[TBD]
w/o ProbPrior	[TBD]	[TBD]	[TBD]
w/o ProbAction	[TBD]	[TBD]	[TBD]
w/o ProbPon	[TBD]	[TBD]	[TBD]

Table 4: Online evolution over 5 rounds (PLACEHOLDER VALUES).

Method	R1 SR	R3 SR	R5 SR	KPR
Base-NoGate	[TBD]	[TBD]	[TBD]	[TBD]
Base-BankCuration	[TBD]	[TBD]	[TBD]	[TBD]
Base-C-ACT	[TBD]	[TBD]	[TBD]	[TBD]

Result Structure (to be filled post-experiment)

The Results section will report: (1) harmful reuse reduction (HRR) across methods, expected pattern of monotonic decrease from Base-NoGate through C-ACT-Full; (2) coverage-risk calibration improvement from adaptive thresholds over FixedBayes; (3) task success gains, expected concentrated on hard tasks (tech-tree, failure-recovery, interaction-stress) where knowledge reuse quality matters most, with null or minimal effect on exploration tasks (positive control); (4) ablation component ranking; (5) online evolution trends showing C-ACT preventing knowledge pollution while no-gate baselines degrade.

Discussion

Why Admission Over Parameterization?

A natural alternative would be to parameterize all candidate knowledge into model weights and rely on optimization to select useful knowledge. We argue this conflates *representability* (can the model encode this knowledge?) with *admissibility* (should it?). Parameterization without admission testing cannot distinguish knowledge that *causes* success from knowledge that merely *accompanies* it. In continual settings, admitting harmful parameter updates causes same-category forgetting that is expensive to reverse. C-ACT’s decision-time admission offers three advantages: (i) reversibility—knowledge can be instantly disabled by lifecycle transition without retraining; (ii) context-dependence—the same knowledge may be safe in one context and harmful in another; (iii) auditability—contract verification and propensity logging provide a complete decision trace.

Design Choices

Independent Beta posteriors. We use independent Beta distributions rather than a joint Dirichlet model. This is de-

liberate: independent Betas are more robust under data sparsity, more computationally efficient ($O(1)$ conjugate updates), and more interpretable. The independence assumption is harmless because decisions only compare p_{use} and p_{base} through $\pi_u(c)$, which is well-defined regardless of their joint distribution.

Probation as supervised reuse. The key design choice in the lifecycle is giving Probation-phase knowledge conditional access rather than full reuse or full rejection. This shrinks the gate’s false-negative rate—good knowledge contributes value during Probation—while preventing the wide-posterior risk that comes with unrestricted Certified access.

Limitations

Contract extraction. Current contract extraction uses keyword-based heuristics. While conservative (erring toward blocking), this can produce false-positive safety boundaries. Future work could use the VLM itself for higher-precision contract generation.

Environment scope. Our evaluation is in Minecraft. While Minecraft is arguably the most complex controllable open-world environment for agent research, validation in other settings would strengthen generality.

Knowledge pipeline dependence. C-ACT requires an upstream generation pipeline. It is purely an admission layer—it cannot create knowledge, only govern its reuse.

Ethics Statement

This work develops a decision-time governance mechanism for AI agents. C-ACT reduces harmful agent behavior (e.g., using the wrong tool for a dangerous operation, consuming irreplaceable resources) by enforcing contract-based safety boundaries and statistically calibrated risk budgets. The system is designed to make agent decisions more auditable through contract verification and propensity logging. We do not foresee direct negative societal impacts from this work;

the primary risk is that a poorly calibrated gate could reject beneficial knowledge, reducing agent capability. This is mitigated by the supervised reuse regime and adaptive calibration. All experiments use simulated Minecraft environments; no real-world physical systems are involved.

Conclusion

We introduced C-ACT, a decision-time admission layer that addresses a structural gap in self-evolving agent design: the absence of causal measurement in knowledge reuse decisions. By transforming knowledge into falsifiable contracts, gating reuse on Bayesian counterfactual uplift with adaptively calibrated risk budgets, and governing knowledge through a six-state lifecycle with supervised reuse, C-ACT provides a principled framework for deciding whether retrieved knowledge should affect agent behavior. The system is lightweight, compatible with any knowledge generation pipeline, and fully auditable.

References

- Angelopoulos, A. N.; and Bates, S. 2021. A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. *arXiv preprint arXiv:2107.07511*.
- Fan, L.; Wang, G.; Jiang, Y.; Mandlekar, A.; Yang, Y.; Zhu, H.; Tang, A.; Huang, D.-A.; Zhu, Y.; and Anandkumar, A. 2022. MineDojo: Building Open-Ended Embodied Agents with Internet-Scale Knowledge. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Lifshitz, S.; Paster, K.; Chan, H.; Ba, J.; and McIlraith, S. 2023. STEVE-1: A Generative Model for Text-to-Behavior in Minecraft. *arXiv preprint arXiv:2306.00937*.
- Lin, H.; Wang, Z.; Ma, J.; and Liang, Y. 2024. MCU: A Comprehensive Benchmark for Minecraft Agents. *arXiv preprint*. Verify exact title and authors before final submission.
- Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023a. Voyager: An Open-Ended Embodied Agent with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Wang, Z.; Cai, S.; Chen, G.; Liu, A.; Ma, X.; and Liang, Y. 2023b. Describe, Explain, Plan and Select: Interactive Planning with LLMs Enables Open-World Multi-Task Agents. In *Advances in Neural Information Processing Systems (NeurIPS)*. Verified via DBLP.
- Wang, Z.; Cai, S.; Liu, A.; Jin, Y.; Hou, J.; Zhang, B.; Lin, H.; He, Z.; Zheng, Z.; Yang, Y.; Ma, X.; and Liang, Y. 2025. JARVIS-1: Open-World Multi-Task Agents with Memory-Augmented Multimodal Language Models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(3): 1894–1907. ArXiv:2311.05997. Verified via DBLP; published venue is IEEE TPAMI 2025, not NeurIPS 2024.